6 Low-level programming

> "Any sufficiently advanced technology is indistinguishable from magic."
>
> *Arthur C. Clarke*

When we learn about low-level computing we learn about processor architectures[1] such as x86 and how to program them. The architecture comprises the instructions and resources that we can use to get work done. We start with learning a mental model where we compute on a single item of data at a time on each processing core. Computing involves moving data between memory and registers, and performing calculations on the contents of registers.[2] In this part of the course we review low-level programming.

1: Also called ISA, for *Instruction Set Architecture*.

2: So-called "complex" architectures like x86 provide instructions that individually carry out *several* (and seemingly independent) tasks—such as obtaining data from memory, computing on it, or conditionally copying memory. Machine codes in x86 can range between 1 and 15 bytes. There are simpler architectures.

Notes

- We start with a review of (parts of) the x86-64 ISA. We spend some time reading and writing assembly code to refresh our understanding.
- We use AT&T syntax—if you prefer Intel syntax than include the "`-masm=intel`" parameter—e.g., in `gcc hello.c -S -masm=intel`.
- To understand x86 assembly programs, it's important to understand the so-called "System V AMD64 ABI"[3], in particular its calling conventions. These conventions establish how we're to use registers when passing parameters to procedure calls and receiving results.
- Also, be sure to recall the memory layout of programs—for instance, the formation and management of stack frames, heap memory, etc—and the ISA's status flags.
- It's also important to understand the forms that the program takes as it changes from C into assembly source code, and from assembly source to compiled assembly, and to linked compiled code. An issue that comes up here is so-called RIP-relative addressing.
- We look at several examples of C programs and what assembly they produce. Examples include: adding int and long, pointers, printing "Hello world", and defining and using pointers. Also, control structures such as loops, calling functions, ternary comparison, if/then/else, and switch. Then we'll look at float (and the difference in generated assembly when we add "`-mavx2`").
- Consdier the following C code:

3: https://en.wikipedia.org/wiki/X86_calling_conventions

Workflow for assembly programming

What workflow and tools do you use for assembly programming? Does it differ from what you use when programming in other languages? If you find assembly programming to be difficult, then what tooling or workflow improvements can help make that programming easier for you?

```c
// Summing two integers
int
summation_int (int x, int y)
{
  return (x + y);
}

// Summing two integers and returning a long
```

```
long
summation_int_to_long (int x, int y)
{
  return ((long)(x + y));
}

// Summing two longs
long
summation_long (long x, long y)
{
  return (x + y);
}
```

Optimization levels

What level of optimization was used for this compilation? Try compiling the code yourself with different -O levels and observe the different outcomes.

- The code shown above compiles to the following:

```
    .file   "1.c"
    .text
    .globl  summation_int
    .type   summation_int, @function
summation_int:
.LFB0:
    .cfi_startproc
    endbr64
    leal    (%rdi,%rsi), %eax
    ret
    .cfi_endproc
.LFE0:
    .size   summation_int, .-summation_int
    .globl  summation_int_to_long
    .type   summation_int_to_long, @function
summation_int_to_long:
.LFB1:
    .cfi_startproc
    endbr64
    addl    %esi, %edi
    movslq  %edi, %rax
    ret
    .cfi_endproc
.LFE1:
    .size   summation_int_to_long, .-summation_int_to_long
    .globl  summation_long
    .type   summation_long, @function
summation_long:
.LFB2:
    .cfi_startproc
    endbr64
    leaq    (%rdi,%rsi), %rax
    ret
    .cfi_endproc
.LFE2:
    .size   summation_long, .-summation_long
    .ident  "GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0"
    .section    .note.GNU-stack,"",@progbits
    .section    .note.gnu.property,"a"
    .align 8
    .long    1f - 0f
    .long    4f - 1f
    .long    5
0:
```

```
    .string  "GNU"
1:
    .align 8
    .long    0xc0000002
    .long    3f - 2f
2:
    .long    0x3
3:
    .align 8
4:
```

- Consider this classical example program:

```
#include <stdio.h>

int
main ()
{
  printf("Hello, world\n");
  return 0;
}
```

- That example compiles to the following:

```
    .file   "2.c"
    .text
.Ltext0:
    .section    .rodata
.LC0:
    .string "Hello, world"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .file 1 "2.c"
    .loc 1 5 1
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    .loc 1 6 3
    leaq    .LC0(%rip), %rdi
    call    puts@PLT
    .loc 1 7 10
    movl    $0, %eax
    .loc 1 8 1
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size   main, .-main
```

- Our workflow consists of inspecting -S output, changing it, and compiling. This'll help us refresh our understanding of using gdb and objdump and nm. For objdump, we see "-t" and "-D" modes.
- For gdb, we'll see the use of the interactive prompt for loading a pro-

Generated assembly

The generated assembly contains a lot of metadata in addition to code! Where does that metadata come from, and what purpose does it serve?

objdump flags

See the objdump manual page to understand the behaviour of the "-t" and "-D" modes.

gram and stepping through source and assembly instructions, and observing the updating of the register values. In particular, we'll review the following gdb commands: set args, run, break & clear, continue & next (skip over calls) & step & stepi, disassembly, info registers, backtrace & frame & up & down, print/[d,a,s], x/[d,l,s]. The most important command is **help** followed by a command you'd like to understand. You're also encouraged to use gdb's tui[4] and its "layout" and "focus" commands.

4: text user interface

- *Intrinsics* will play a key part of our study of vector computing. An intrinsic maps to one or more assembly instructions. Recall the example we developed a couple lectures back to measure the overhead of certain functions by using the *time stamp counter*[5]:

5: https://www.intel.com/content/www/us/en/docs/trace-analyzer-collector/user-guide-reference/2021-10/cpu-cycle-counter.html

```
#include "omp.h"
#include "x86intrin.h"
#include "stdlib.h"
#include "stdio.h"

#define COUNT 10000

int
main ()
{
unsigned long start, end;

start = __rdtsc();
for (int i = 0; i < COUNT; i++)
  omp_get_wtime();
end = __rdtsc();

printf("Function took %lu cycles\n", ((end - start) / COUNT
    ));
}
```

In that example, we're relying on an intrinsic to invoke the RDTSC instructions.

Exercise

Rewrite the C program to use *inline assembly* instead of intrinsics.

- Refreshing our understanding of the x86 ISA also provides an opportunity to learn about architecture-provided *synchronization primitives*—such as the CMPXCHG[6] instruction for concurrent and parallel[7] computations. Using those primitives you can build (architecture-specific but) language-level concurrency abstractions and libraries such as those provided by pthreads and OpenMP.

6: https://www.felixcloutier.com/x86/cmpxchg

7: The LOCK prefix must be used in a parallel setting—i.e., where there's more than one core. CMPXCHG was introduced on single-core targets to improve their support for multi-tasking.

Exploring the instruction set

Browse the instruction set for interesting instructions—e.g., RDSEED.

- See Agner Fog's tables[8] to find out the approximate *cycle count* of an instruction.[9]

8: https://www.agner.org/optimize/instruction_tables.pdf

9: See also the fantastic work by Abel et al. at https://uops.info/